

Lab Report 2: Delta Modulation and Pulse Code Modulation in Data Communication

Pulse Code Modulation (PCM)

Objective

To understand and implement Pulse Code Modulation (PCM) for converting analog signals into digital signals, and analyze the effects of sampling, quantization, and encoding on signal quality.

Apparatus

1. Function Generator
2. Oscilloscope
3. Analog-to-Digital Converter (ADC)
4. Digital-to-Analog Converter (DAC)
5. Personal Computer with PCM software
6. MATLAB/Python for simulation

Theory

PCM involves three steps:

1. **Sampling:** Analog signals are sampled at intervals $\geq$ twice the highest frequency (Nyquist rate).
2. **Quantization:** Sampled values are approximated to discrete levels, introducing quantization error.
3. **Encoding:** Quantized values are converted to binary for digital transmission.

Procedure

1. Generate a 1 kHz sine wave using a function generator.
2. Sample the signal via ADC at ≥ 2 kHz.
3. Quantize samples to discrete levels.
4. Encode quantized values into binary.
5. Transmit the digital signal.
6. Reconstruct the signal using DAC and observe using an oscilloscope.

Simulation Procedure

1. Generate a sine wave in Python.

2. Simulate sampling, quantization, and encoding.
3. Reconstruct and plot original vs. reconstructed signals.

Experimental Code & Graph

python

```
import numpy as np
import matplotlib.pyplot as plt

t = np.arange(0, 20, 0.005)
partition = np.arange(-1, 2, 0.2)
codebook = np.arange(-1, 2.2, 0.2)
x = np.sin(t)
index = np.digitize(x, partition)
quants = codebook[index-1]

# Plotting
plt.subplot(3, 1, 1)
plt.plot(t, x)
plt.title('Message Signal')
plt.xlabel('Time(s)')
plt.ylabel('Amplitude(V)')

plt.subplot(3, 1, 2)
plt.plot(t, quants)
plt.title('Quantized Signal')

plt.subplot(3, 1, 3)
y = np.uint8(index - 1)
plt.plot(t, y)
plt.title('PCM Signal')

plt.tight_layout()
plt.show()
```

Output Waveforms

- **Message Signal:** Sine wave (1 kHz, 20s duration).
- **Quantized Signal:** Stepped approximation of sine wave.
- **PCM Signal:** Binary-encoded pulses.

Discussion

PCM is foundational for digital communication, enabling efficient signal transmission. The lab demonstrated the impact of sampling rate and quantization levels on reconstruction accuracy. Higher bit depths reduce quantization noise, validating PCM's relevance in high-fidelity systems.

Delta Modulation (DM)

Objective

To implement Delta Modulation (DM), analyze its performance, and evaluate quantization noise and slope overload effects.

Apparatus

1. Delta Modulation Trainer Kit
2. Oscilloscope
3. Function Generator
4. MATLAB/Python for simulation

Theory

DM encodes the difference between successive samples into a 1-bit stream. Key steps:

1. **Sampling:** Regular intervals.
2. **Quantization:** Difference quantized to $\pm\Delta$.
3. **Encoding:** Binary output (1 for $+\Delta$, 0 for $-\Delta$).
4. **Integration:** Reconstruction via accumulated steps.

Procedure

1. Generate a low-frequency sine wave using a function generator.
2. Transmit through DM modulator.
3. Observe modulated output (staircase waveform) on an oscilloscope.
4. Demodulate and compare reconstructed vs. original signal.
5. Repeat for varying frequencies/amplitudes to study noise and distortion.

Simulation Procedure

1. Generate sine and sinc signals in Python.
2. Implement DM encoding/decoding.
3. Plot original, modulated, and demodulated signals.

Experimental Code & Graph

python

```

import numpy as np
import matplotlib.pyplot as plt

t = np.arange(0, 1.01, 0.01)
d = 2 * np.pi / 100 # Step size

# Sinc pulse DM
m = np.sinc(2 * np.pi * t)
mq = np.zeros_like(m)
e = np.zeros_like(m)

for n in range(100):
    if n == 0:
        e[n] = m[n]
    else:
        e[n] = m[n] - mq[n-1]
    eq = d * np.sign(e[n])
    if n == 0:
        mq[n] = eq
    else:
        mq[n] = mq[n-1] + eq

# Sine wave DM
m1 = np.sin(2 * np.pi * t)
mq1 = np.zeros_like(m1)
e1 = np.zeros_like(m1)

for n in range(100):
    if n == 0:
        e1[n] = m1[n]
    else:
        e1[n] = m1[n] - mq1[n-1]
    eq1 = d * np.sign(e1[n])
    if n == 0:
        mq1[n] = eq1
    else:
        mq1[n] = mq1[n-1] + eq1

# Plotting
plt.subplot(2, 1, 1)
plt.plot(m, 'k*', label='Original Sinc')
plt.plot(mq, 'red', drawstyle='steps-post', label='DM Output')
plt.title('Sinc Pulse DM')

```

```
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(m1, 'red', label='Original Sine')
plt.plot(mq1, 'blue', drawstyle='steps-post', label='DM Output')
plt.title('Sine Wave DM')
plt.legend()

plt.tight_layout()
plt.show()
```

Output Waveforms

- **Sinc Pulse DM:** Staircase approximation tracking the sinc pulse.
- **Sine Wave DM:** Stepwise reconstruction of the sine wave with visible quantization noise.

Discussion

DM's simplicity and bandwidth efficiency make it suitable for voice transmission. However, trade-offs exist: smaller step sizes reduce granular noise but risk slope overload during rapid signal changes. The lab highlighted how step size optimization balances SNR and distortion, critical for practical applications like telephony.

Conclusion

Both PCM and DM are vital in digital communication. PCM offers high fidelity at the cost of bandwidth, while DM prioritizes simplicity and efficiency. The experiments underscored their operational principles, trade-offs, and real-world relevance.